2019 19th International Conference on Advanced Robotics (ICAR)
Belo Horizonte, Brasil, December 02-06, 2019

SimSLAM 2D: A Simulation Framework for Testing and Benchmarking of two-dimensional Visual-SLAM Methods*

Juan Rodriguez[1] and Davinson Castano-Cano[1]

Abstract— **Localization proves still a challenging task in robotics, with Visual-SLAM being a both powerful and popular solution. In Ground-SLAM, a branch of Visual-SLAM that uses groundplane views, there is still a lot to explore but the tools for testing new methods need improvement. In this paper, we present a novel framework for simulating the motion of Visual-SLAM methods over a given groundplane. It can also be used to benchmark different methods on a given dataset. We present the functioning of our system, as well as the requirements for a similar system to work. In order to showcase the capabilities of SimSLAM 2D, we present a test case of different trajectories on a smooth concrete terrain comparing the performance of two methods, StreetMap and ORB-SLAM. We also test StreetMap in localization only mode with the purpose of showing the visualization tools. We thereby show the ease of simulating and testing a Visual-SLAM method on a 2D environment in our framework, which represents a useful tool when developing new Visual-SLAM methods.**

I. INTRODUCTION

In mobile robotics, localization is one of the main topics of interest, as most control actions require knowledge of the robot position and orientation. Among the possible solutions to this problem, visual SLAM has been an extensively used technique. In it, a map is first created from sensor data (e.g. RGB or RGB-D images) to which later acquired data is then compared against, thus achieving localization.

However, the usual approach of having front- or side-facing cameras poses a problem when dealing with highly dynamical environments such as parking lots, storage units with varying inventory or modular production systems. In these, the robot can not confidently rely on the map it creates for online localization as the surrounding space is changing constantly. In order to overcome this issue, alternative approaches such as ceiling view or groundplane view SLAM (CV-SLAM and Ground-SLAM , respectively) have been proposed. CV-SLAM [1], [2], [3] presents an upward-facing camera that takes pictures of the ceiling in order to get data that is less variant in time, whereas in Ground-SLAM [4], [5], [6], [7] a downward-facing camera is used for the same purpose. CV-SLAM presents a challenge that is difficult to overcome, which is the lighting on the ceiling.

An issue arises when trying to develop a method that uses such a system, that is, there is no unified method to test and compare them. For testing a new method, a researcher has to resort to field experiments that require time and equipment. Measuring instruments to get ground truth measurements (such as DGPS, Leica trackers[2], or Ultra Wide Band systems) can be expensive and can only test methods that are as precise as its measurements, making it considerably more difficult to test methods that are intended to work in indoor environments (see e.g. [8], [9] for reported accuracies in indoor systems) and/or want to achieve global localization.

In this work, we present a framework called SimSLAM 2D that addresses both of these issues. It gives the user the ability to easily test any two-dimensional visual SLAM method that takes images as input, as well as the option to benchmark different methods and compare them, providing an exact value for the ground truth on both online and offline localization. Our framework can also generate image sequences to be used as training data in algorithms that need training.

The remainder of this paper is organized as follows: in Section II the related work is presented. Then, Section III describes the requirements of a 2D vision-based SLAM simulator and tester. Afterwards, in Section IV our system is showcased together with an analysis of the features it presents. In section V, we present the details of our implementation. Finally, Section VII shows a hypothetical test case comparing StreetMap [10] and ORB-SLAM [11], two state-of-the-art methods (one in Ground-SLAM and another one in regular visual SLAM, respectively).

II. RELATED WORK

A. 3D RGB-D Benchmarks

There exist multiple RGB-D datasets available for benchmarking of Visual SLAM methods. Some commonly used ones are the TUM Dataset [12], the KITTI Dataset[13] and the EuRoC Dataset [14]. These range from hand-held applications [15], [12], [16], to vehicles [14], [13], [17], to robotic systems [12]. Together with these multiple error measures such as the Relative Pose Error (RPE) or the Absolute Trajectory Error (ATE) have been proposed [12]. Finally, multiple benchmarking frameworks (see e.g. [18] or [19]) that can incorporate different datasets have been developed.

Nevertheless, none of these datasets is directly applicable to a method made for two-dimensional robotic motion, as they are made for three-dimensional movement or use 3D Data. Others, such as the KITTI Dataset, provide information only on front- or backward-facing cameras.

*This work was supported by EAFIT University and the Vicerrectoría de Descubrimiento y Creación

[1]J. Rodriguez and D. Castano-Cano are with the Operations and Logistics Group from the Production Engineering Department, School of Engineering, EAFIT University, Carrera 49 No 7Sur-50, Medellin, Colombia. jrodri56@eafit.edu.co dcasta25@eafit.edu.co

[2]https://leica-geosystems.com/

Authorized licensed use limited to: Hochschule Emden/Leer. Downloaded on November 17,2025 at 13:20:18 UTC from IEEE Xplore. Restrictions apply.

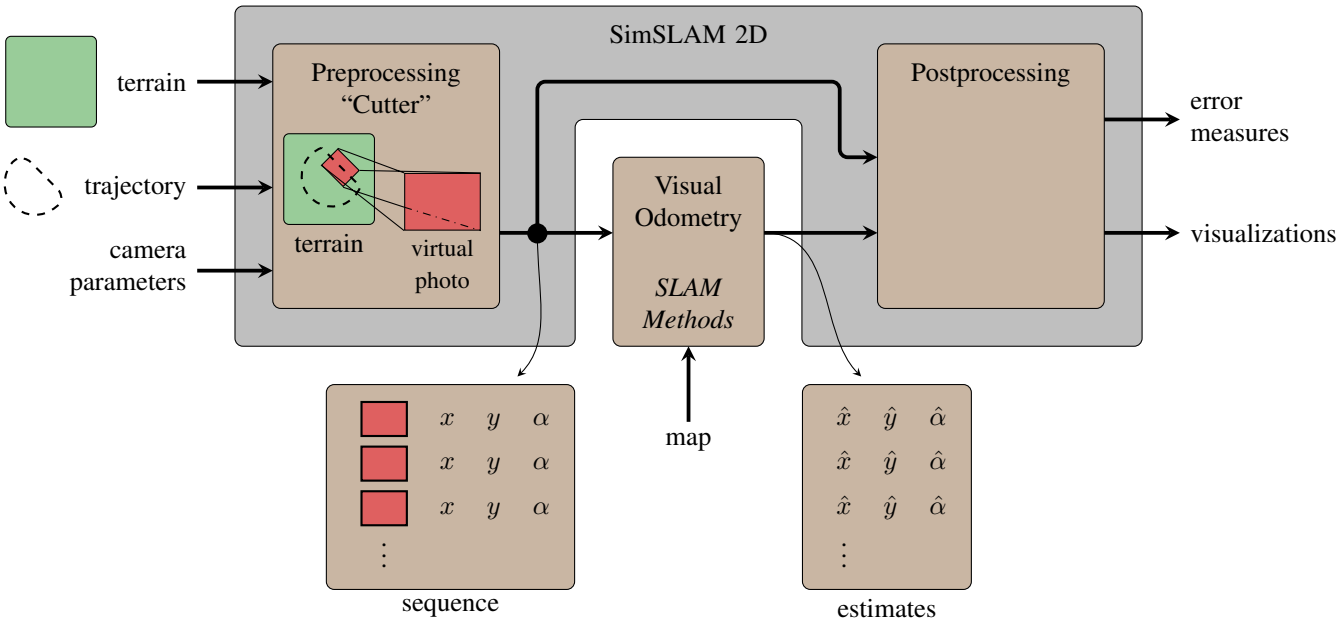


Fig. 1. SimSLAM 2D workflow.

B. 2D Visual SLAM Benchmarking

To the best of our knowledge, no framework for simulation or benchmarking two-dimensional vision-based SLAM methods exists so far. However in [20], the authors present a simulation based benchmarking framework of laser-based SLAM methods in which variation is introduced by introducing sensor error in multiple runs of different environments, aiming to achieve repeatability of the experiments used to benchmark these methods. Note that we are not presenting a specific dataset for benchmarking, but rather a framework that can use any properly formatted dataset to test methods with it.

III. REQUIREMENTS

Here we present the requirements for a framework that simulates and/or tests a two-dimensional SLAM method. We do it in a general way so it can be applied to different methodologies of constructing such a framework. Additional features such a system could have are also mentioned.

A 2D SLAM simulation framework must be able to:

- Provide the method to test/simulate with a sequence of images. The framework must be able to know the correct position and orientation associated with each provided image.
- Test the method with different sequences.
- Provide the user with at least one error measure between the real trajectory associated with the provided image sequence and the one predicted by the method.
- Test/simulate different methods.
- Generate sequences from different sources, that is, different textures, environments, lighting conditions, etc.

Some desirable features are:

- Visualizing the provided/predicted sequence of images and the difference between both.
- Being able to apply multiple error measures on a given sequence.
- Saving sequences of images together with the ground truth position to use them in models that require training.
- Provide data augmentation to enhance sequences of images [21], such as simulated lighting conditions or motion blur [22].

IV. DESCRIPTION

Fig. 1 showcases the workflow of our system. SimSLAM 2D has three input parameters:

1) **The terrain image**, which is a high-resolution panoramic picture of the environment that the user wishes to simulate, it can be taken with the intended camera the method will work with or a different camera (as it is explained further on, cameras can also be simulated).
2) **The trajectory**, which is a set of camera poses defined by the coordinates x, y and orientation α of each pose, which are given with respect to a fixed coordinate system in the terrain image.
3) **The camera parameters**, which are the group of parameters that define the virtual camera, these include, but are not limited to, the resolution, the intrinsic matrix and warping factors of the camera.

With these three inputs, SimSLAM2D performs two processes that encapsulate visual-based SLAM Methods. The processes are described as follow:

1) Before using a visual-based SLAM Method, SimSLAM2D passes the inputs through the pre-processing

Authorized licensed use limited to: Hochschule Emden/Leer. Downloaded on November 17,2025 at 13:20:18 UTC from IEEE Xplore. Restrictions apply.

unit (nicknamed "Cutter"). This unit performs different steps:

- It takes virtual photos of the input terrain following the input trajectory. These photos take into account the camera parameters.
- Then, it appends the virtual photo with the corresponding pose (x, y, α) into a sequence array.
- After that, it pipelines the sequence array into the visual-based SLAM Method.

2) After the visual-based SLAM Method gives predicted poses. The postprocessing unit compares them with the sequence array. The comparison process produces visualizations and error measures.

Note that the Visual odometer can optionally take a previously generated Map in case the method is being tested in localization only mode.

V. IMPLEMENTATION DETAILS

The framework runs on Python, making use of OpenCV to handle operations on the terrain image and the virtual photos. We use NumPy for pre-processing trajectories (obtaining orientations from x, y pairs, scaling, rotating, etc.), computing error estimates, and transforming the resulting trajectory estimates before error estimation.

Our StreetMap implementation also runs on Python, using OpenCV and NumPy for some computations and Python bindings of the graph optimization library g2o. Similarly, for testing ORB-SLAM, we used Python bindings of the implementation released by the authors.

Regarding our approach, having a large terrain image represents two practical problems. The first one corresponds to the computational cost of rotating big pictures in order to obtain image crops in the desired orientations. The second one corresponds to the memory needed to operate on a large scale image such as the terrain one.

To overcome these issues, we propose the following procedures:

For **cropping**, we first find the region of interest (ROI) of the resulting crop, this region is cropped and then rotated to the desired position. Here we recalculate the desired crop area in this orientation and perform the final crop. In summary, the process consists of two crops and a rotation, where the first crop serves the purpose of reducing the size of the image to be rotated in the rotation step. The calculation of the ROI is cheaply carried out from the height, width, center, and orientation of the rectangle to crop.

In order to reduce **memory requirements**, we store the terrain image in smaller image files in a grid-like fashion. We then load these files as required by the aforementioned ROI.

VI. DATA GENERATION

As previously stated, our framework makes use of a terrain image as the primary data source. Here we explain how to generate such an image and what are the characteristics it must have.

The data acquisition method requires three steps:

1) A camera pointing down, preferably with a stabilizing device such as a gimbal, is mounted on a moving platform (or a robot with planar motion).
2) The camera takes pictures or a video of the terrain to be simulated. They should be taken on a single run to reduce variability, especially in environments with conditions that vary in time, such as lighting when using natural light.
3) Finally, the pictures are stitched together using a panorama stitching software. We note that panorama stitching programs that can identify planar motion and take advantage of this fact produced the best results for our test.

In this manner, our framework can work with any 2D panoramic image generated by a camera following planar motion. Some considerations to take into account are:

- Even though recording a video is usually much faster than taking multiple pictures at strategic positions, it results in motion blur on the resulting images. This can be alleviated by the panorama stitching procedure or by limiting the moving speed while recording.
- High reflectance of the terrain may result in images that are hard to stitch, as well as introduce artificial artifacts not present in the real world. Special care has to be taken with these terrains, as the algorithms used in simulations can mistakenly use these artifacts as features when they are not present in the real-life environment.
- Even though terrain planarity is important (as the framework and some of the algorithms assume planar motion), slight irregularities are acceptable provided images are obtained in a way that closely resembles real-life robot operation conditions.

VII. USE CASE

A. Data

The methods were tested on a single terrain image, showcased in Fig. 2. The terrain corresponds to a smooth concrete floor, which at the moment of capture had the lighting conditions illustrated in Fig. 3. The terrain was indirectly illuminated on the left side by sunlight that passed through a glass door, fading into indoor lighting provided by halogen lamps. We chose this illumination with the purpose of having different testing conditions on a single terrain image.

The terrain image was obtained from a set of pictures from a Canon 1100D camera on video mode (resolution of 1280 by 720) with a **EF-S 18-55 mm f/3.5-5.6 III**[3] lens. The camera sensor was parallel to the floor at **45 cm** above it. The pictures were then stitched using Microsoft ICE[4], resulting in one 17706 by 4370 pixels image, 77.4 Megapixels and an aspect ratio of 4.05.

[3]https://www.canon.com.au/camera-lenses/ef-s-18-55mm-f-3-5-5-6-iii

[4]https://www.microsoft.com/en-us/research/product/computational-photography-applications/image-composite-editor/

Authorized licensed use limited to: Hochschule Emden/Leer. Downloaded on November 17,2025 at 13:20:18 UTC from IEEE Xplore. Restrictions apply.

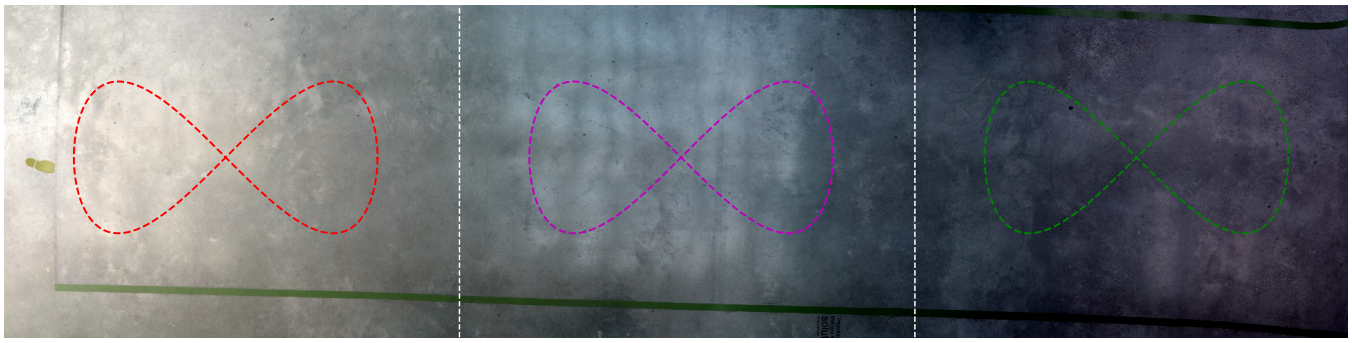


Fig. 2. Terrain map partitioned in three equal parts, with different lighting conditions. From left to right, high, medium and low lighting conditions. Each part presents a trajectory that will be used later in this paper.

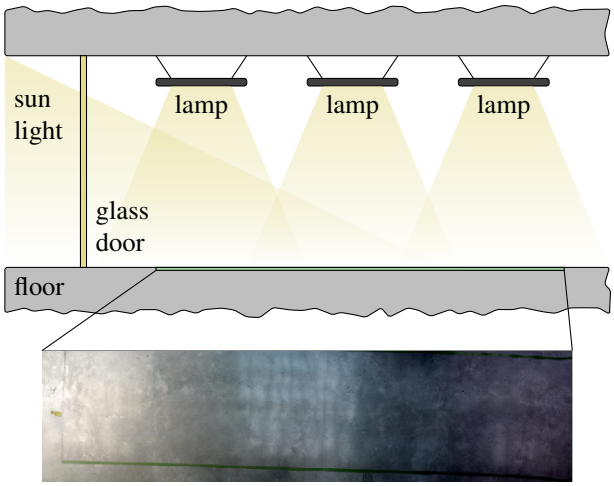


Fig. 3. Lighting conditions of the terrain.

B. SLAM Methods

We consider two methods for testing our framework:

1) **StreetMap** [10], uses SURF features [23] to find the relative motion between two consecutive frames, assuming plane motion and performing pose graph optimization in order to minimize the cumulative drift error.
2) **ORB-SLAM** [11], on the other hand, makes use of ORB features without necessarily assuming plane motion, together with different techniques that help improve efficiency.

Please note that this is not an exhaustive comparison of the two methods; as no Open-Source implementation of StreetMap exists, we used our own implementation of it so results may differ from the method presented in their work. Results are given for illustrative purposes only.

C. Tests

Two tests were then conducted on this terrain, the first one comparing the two methods on specific parts of the terrain and the second one showing the reconstruction of a trajectory on a wider part of the terrain.

The first test consists of three closed trajectories, each of them discretized using 500 points (i.e. 500 poses in the sequence) and covering a part of the terrain that has approximately the same lighting conditions. This is done in order to test the difference under each of the light levels achieved (in Fig. 2: High, Medium and Low light from left to right.) The resulting sequences were then passed to each of the SLAM Methods and the estimates were compared with the ground truth using the Root mean squared error (RMSE) of the Absolute Trajectory Error (ATE) as defined by [12] and a graphical comparison. The number of poses was chosen for ORB-SLAM to have enough data to achieve a proper initialization. For ORB-SLAM we use the same hyperparameters for the TUM examples the authors provide.

In the second test, we used only StreetMap in localization-only mode. To form the initial map, the method was provided with a sequence of a zig-zag trajectory that covered the whole terrain (here not shown). Afterwards, a sequence of 250 poses was tested for localization on this map. The trajectory formed by this sequence together with the contour of the seen area can be seen in Fig. 5, this trajectory occupied approximately half of the terrain. Finally, the trajectory was reconstructed by taking the provided images and placing them on the position estimated by the method, ideally reproducing the image encapsulated by the previously mentioned contour.

D. Results

We present the sequence estimates of the first test on Fig. 4, it can be seen that there is an overall qualitative agreement between both methods and the ground truth. We report the RMSE for every trajectory in Table I.

According to these results, it is clear that StreetMap performed better in all three trajectories. This could be attributed, according to their paper, to the fact that StreetMap assumes planar motion. Interestingly, the Medium trajectory was where StreetMap performed best and ORB-SLAM

TABLE I

RMSE IN METERS FOR THE DIFFERENT TRAJECTORIES.

	High [m]	Medium [m]	Low [m]
StreetMap	$9.13\text{x}10^{-3}$	$7.82\text{x}10^{-3}$	$9.10\text{x}10^{-3}$
ORB-SLAM	$10.62\text{x}10^{-3}$	$10.82\text{x}10^{-3}$	$9.58\text{x}10^{-3}$

Authorized licensed use limited to: Hochschule Emden/Leer. Downloaded on November 17,2025 at 13:20:18 UTC from IEEE Xplore. Restrictions apply.

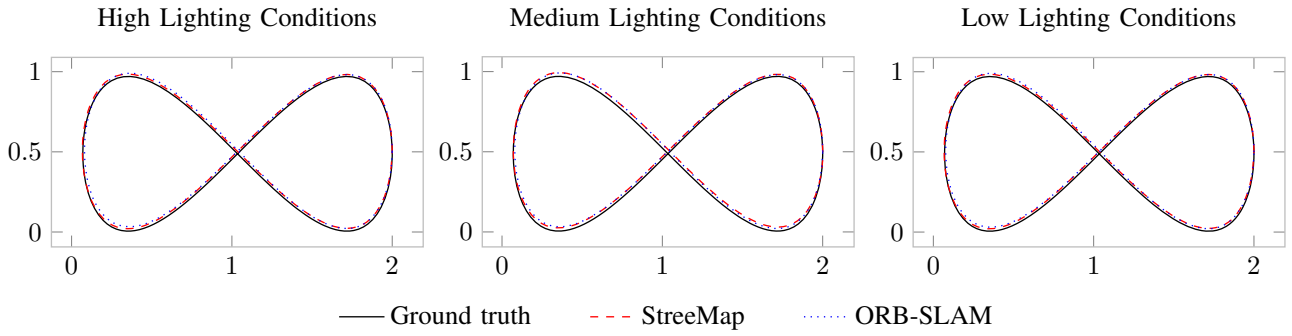


Fig. 4. Trajectory results for the first test. Each plot corresponds to a different lighting condition: High, Medium and Low, respectively. Units in meters.

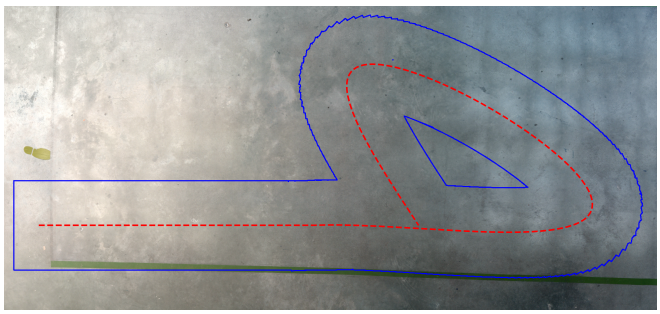


Fig. 5. Trajectory and contour for the second test.



Fig. 6. Trajectory reconstruction.

worst, this may be due to the presence of lamp reflections present in this part of the terrain.

The relatively small error is to be expected given the high number of images provided in the sequence, implying a wide area of intersection between subsequent images, as well as the lack of motion blur and small area covered by the trajectory.

For the second test, we show the reconstruction of the localization trajectory in Fig. 6. No major discrepancies are present between it and the expected contour (Fig. 5), this can be easily seen in the borders of the contour or the straight green line at the bottom of the trajectory.

VIII. Conclusions

We present the SimSLAM 2D framework which allows us to test and benchmark different Visual-SLAM methods in two dimensions. This framework is the first of a kind with a focus on 2D environments.

The use of the framework gives the ability to easily test different Visual-SLAM methods and compare them. The comparison can give information on which method to use in a specific scenario.

With the framework, we can compare to a ground truth without uncertainties, which allow to develop and test different SLAM methods without the need for expensive measuring equipment.

From the results obtained in the benchmark of the StreetMap [10] and ORB-SLAM [11], we found that StreetMap performs better than ORB-SLAM in the task that it is designed for (see table I).

To improve the framework, we are going to implement realistic conditions including: motion blur, lighting condition changes, among others. We also plan on making the framework Open-Source and available to the research community.

We are going to use this framework to compare different Visual-SLAM methods and to propose new ones that perform better in Ground-SLAM scenarios similar to the test case in this work.

References

[1] L. Li, H.-R. Kim, S. Jiang, Y.-S. Kim, and T.-Y. Kuc, "Feature saliency based slam of mobile robot," in *2018 International Conference on Electronics, Information, and Communication (ICEIC)*. IEEE, 2018, pp. 1–3.

[2] W. Y. Jeong and K. M. Lee, "CV-SLAM: A new ceiling vision-based SLAM technique," in *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS*, 2005.

[3] H. Choi, S. Jo, and E. Kim, "Cv-slam using line and point features," in *2012 12th International Conference on Control, Automation and Systems*. IEEE, 2012, pp. 1465–1468.

[4] N. Nourani-Vatani and P. V. K. Borges, "Correlation-based visual odometry for ground vehicles," *Journal of Field Robotics*, vol. 28, no. 5, pp. 742–768, 2011.

[5] H. Fang, M. Yang, R. Yang, and C. Wang, "Ground-texture-based localization for intelligent vehicles," *IEEE Transactions on Intelligent Transportation Systems*, vol. 10, no. 3, pp. 463–468, 2009.

[6] R. Gonzalez, F. Rodriguez, J. L. Guzman, C. Pradalier, and R. Siegwart, "Combined visual odometry and visual compass for off-road mobile robots localization," *Robotica*, vol. 30, no. 6, pp. 865–878, 2012.

[7] A. Kelly, "Mobile robot localization from large-scale appearance mosaics," *The International Journal of Robotics Research*, vol. 19, no. 11, pp. 1104–1125, 2000.

[8] A. F. G. Ferreira, D. M. A. Fernandes, A. P. Catarino, and J. L. Monteiro, "Localization and positioning systems for emergency responders: A survey," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2836–2870, 2017.

Authorized licensed use limited to: Hochschule Emden/Leer. Downloaded on November 17,2025 at 13:20:18 UTC from IEEE Xplore. Restrictions apply.

[9] A. Hameed and H. A. Ahmed, "Survey on indoor positioning applications based on different technologies," in *2018 12th International Conference on Mathematics, Actuarial Science, Computer Science and Statistics (MACS)*. IEEE, 2018, pp. 1–5.

[10] X. Chen, A. S. Vempati, and P. Beardsley, "Streetmap-mapping and localization on ground planes using a downward facing camera," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 1672–1679.

[11] R. Mur-Artal and J. D. Tardós, "Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras," *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.

[12] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of rgb-d slam systems," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 573–580.

[13] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The kitti dataset," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.

[14] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, "The euroc micro aerial vehicle datasets," *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.

[15] A. Handa, T. Whelan, J. McDonald, and A. J. Davison, "A benchmark for rgb-d visual odometry, 3d reconstruction and slam," in *2014 IEEE international conference on Robotics and automation (ICRA)*. IEEE, 2014, pp. 1524–1531.

[16] O. Wasenmüller, M. Meyer, and D. Stricker, "Corbs: Comprehensive rgb-d benchmark for slam using kinect v2," in *2016 IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 2016, pp. 1–7.

[17] O. Wulf, A. Nuchter, J. Hertzberg, and B. Wagner, "Ground truth evaluation of large urban 6d slam," in *2007 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2007, pp. 650–657.

[18] B. Bodin, H. Wagstaff, S. Saecdi, L. Nardi, E. Vespa, J. Mawer, A. Nisbet, M. Luján, S. Furber, A. J. Davison *et al.*, "Slambench2: Multi-objective head-to-head benchmarking for visual slam," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 1–8.

[19] R. Kümmerle, B. Steder, C. Dornhege, M. Ruhnke, G. Grisetti, C. Stachniss, and A. Kleiner, "On measuring the accuracy of slam algorithms," *Autonomous Robots*, vol. 27, no. 4, p. 387, 2009.

[20] F. Amigoni, V. Castelli, and M. Luperto, "Improving repeatability of experiments by automatic evaluation of slam algorithms," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 7237–7243.

[21] L. Perez and J. Wang, "The effectiveness of data augmentation in image classification using deep learning," *arXiv preprint arXiv:1712.04621*, 2017.

[22] H. Zhao, H. Shang, and G. Jia, "Simulation of remote sensing imaging motion blur based on image motion vector field," *Journal of Applied Remote Sensing*, vol. 8, no. 1, p. 083539, 2014.

[23] H. Bay, T. Tuytelaars, and L. Van Gool, "Surf: Speeded up robust features," in *European conference on computer vision*. Springer, 2006, pp. 404–417.

Authorized licensed use limited to: Hochschule Emden/Leer. Downloaded on November 17,2025 at 13:20:18 UTC from IEEE Xplore. Restrictions apply.